

Inventory Management System for B2B SaaS

Overview

You're joining a team building "StockFlow" - a B2B inventory management platform. Small businesses use it to track products across multiple warehouses and manage supplier relationships.

Part 1: Code Review & Debugging

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # Create new product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )

    db.session.add(product)
    db.session.commit()

    # Update inventory count
    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

Issues Identified:

1. **No input validation** : Missing or invalid fields may cause runtime errors.
2. **SKU uniqueness not checked**: Duplicate SKUs can be inserted.
3. **Partial Commit**: Partial commits can lead to inconsistent state.
4. **Improper warehouse modeling** : Product should not be tied to a single warehouse.
5. **No error handling** : Failures are not gracefully handled.

6. **Price issues** : Price should be positive and decimal value
7. **No response standardization or HTTP status codes**

Impact On Production:

Issue	Impact
Missing validation	Corrupt or incomplete data
Duplicate SKU	Inventory conflicts, reporting errors
Partial commits	Inconsistent DB state

Fixed Version:

```
@app.post("/api/products", status_code=201)
def create_product(data: ProductCreate):
    try:
        # Check SKU uniqueness
        existing = Product.query.filter_by(sku=data.sku).first()
        if existing:
            raise HTTPException(status_code=409, detail="SKU already exists")

        # Create product
        product = Product(
            name=data.name,
            sku=data.sku,
            price=data.price
        )

        db.session.add(product)
        db.session.flush() # ensures product.id is available before commit

        # Create inventory record
        inventory = Inventory(
            product_id=product.id,
            warehouse_id=data.warehouse_id,
            quantity=data.initial_quantity
        )

        db.session.add(inventory)

        # Commit both operations together
        db.session.commit()

    return {
        "message": "Product created successfully",
        "product_id": product.id
    }
```

```
}

except SQLAlchemyError:
    db.session.rollback()
    raise HTTPException(status_code=500, detail="Database error occurred")

except Exception:
    db.session.rollback()
    raise HTTPException(status_code=500, detail="Unexpected server error")
```

Fix Summary

- Used transaction-safe logic to prevent partial commits
 - Added SKU uniqueness validation
 - Ensured clean error handling
 - Used request schema validation
 - Maintained database consistency
-

Part 2: Database Design

Entities and Relationships:

1. Company Table:

Fields:

Id (Primary Key)
Name(String)
Created_at (DateTime)

Relationships:

One Company Can Have Many Warehouses.
One to many (company to warehouse).

2. Warehouse Table:

Fields:

- Id (Primary Key)
- Company_id (Foreign Key)
- Name (String)
- Location (String)

Relationships:

- One Company Can Have Many Warehouses.
- Many to One (Warehouse to Company)
- Each warehouse stores inventory separately

3. Product Table:

Fields:

- Id (Primary Key)
- Name (String)
- sku (UNIQUE)
- Price (Decimal)
- is_bundle (BOOLEAN)
- low_stock_threshold(Integer)

Relationships:

- SKU must be unique across platform
- is_bundle helps identify combo products
- Product definition is separate from inventory

4. Inventory Table:

Fields:

- Id (Primary Key)
- product_id (FK)
- warehouse_id (FK)
- quantity

Relationships:

- Handles *many-to-many relationship*
- Product to Warehouse (*many-to-many relationship*)
- Quantity varies per warehouse

5. Inventory History Table:

Fields:

- id (PK)
- product_id (FK)
- warehouse_id (FK)
- change_quantity(Integer)
- reason(String)
- timestamp(auto)

Relationships:

- Tracks stock changes (sale, restock, damage)
- Needed for analytics and auditing
- Required for "recent sales" logic

6. Supplier Table:

Fields:

- Id (Primary Key)
- name(String)

- Contact_Email (String)

Relationships:

- Stores supplier information
- Used for reordering alerts

7. ProductSupplier (Many-to-Many):

Fields:

- Id (Primary Key)
- product_id (FK)
- supplier_id (FK)

Relationships:

- One supplier can supply many products
- One product can have multiple suppliers

8. Bundle Mapping Table:

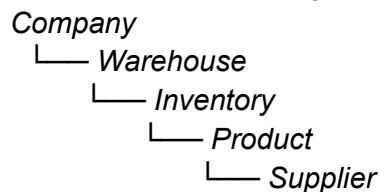
Fields:

- bundle_id (FK → Product.id)
- child_product_id (FK → Product.id)
- quantity

Relationships:

- Supports bundles made of multiple products
- Helps deduct inventory correctly

9. Relationship Summary (Simple View)



Assumptions Made:

Assumptions:

- A warehouse belongs to only one company
 - A product can exist in multiple warehouses
 - Products can have multiple suppliers
 - Low-stock threshold is defined per product
 - Inventory movement history is required for analytics
-

Part 3: API Implementation

Implement an endpoint that returns low-stock alerts for a company.

Business Rules:

- Low stock threshold varies by product type
- Only alert for products with recent sales activity
- Must handle multiple warehouses per company
- Include supplier information for reordering

Endpoint Specification:

GET /api/companies/{company_id}/alerts/low-stock

Assumptions Made

1. A company can have multiple warehouses.
2. A product can exist in multiple warehouses.
3. Each product has a predefined low_stock_threshold.
4. "Recent sales" means at least one sale in the last N days (assume 30 days).
5. Sales data is derived from InventoryHistory.
6. Each product has at least one supplier.
7. One warehouse belongs to only one company.
8. This endpoint is read-only (no mutations).
9. Authentication is assumed to be handled upstream.

API Implementation (Python – Framework-Agnostic FastAPI Like):

```
@app.get("/api/companies/{company_id}/alerts/low-stock")
def get_low_stock_alerts(company_id: int):
    alerts = []

    # 1. Validate company
    company = get_company(company_id)
    if not company:
        return {"error": "Company not found"}, 404

    # 2. Get warehouses for company
    warehouses = get_warehouses_by_company(company_id)

    for warehouse in warehouses:

        inventory_items = get_inventory_by_warehouse(warehouse.id)

        for item in inventory_items:
            product = get_product(item.product_id)
```

```

# 3. Check recent sales (last 30 days)
recent_sales = get_recent_sales(
    product_id=product.id,
    warehouse_id=warehouse.id,
    days=30
)

if recent_sales == 0:
    continue # skip products with no recent activity

# 4. Check low stock condition
if item.quantity < product.low_stock_threshold:

    supplier = get_supplier_for_product(product.id)

    # Optional: calculate stock run-out estimate
    avg_daily_sales = recent_sales / 30
    days_until_stockout = (
        int(item.quantity / avg_daily_sales)
        if avg_daily_sales > 0 else None
    )

    alerts.append({
        "product_id": product.id,
        "product_name": product.name,
        "sku": product.sku,
        "warehouse_id": warehouse.id,
        "warehouse_name": warehouse.name,
        "current_stock": item.quantity,
        "threshold": product.low_stock_threshold,
        "days_until_stockout": days_until_stockout,
        "supplier": {
            "id": supplier.id,
            "name": supplier.name,
            "contact_email": supplier.contact_email
        }
    })

return {
    "alerts": alerts,
    "total_alerts": len(alerts)
}

```

Steps:

1. Validate company existence
2. Fetch all warehouses for the company

3. For each warehouse:
 - Fetch inventory items
 - Check recent sales activity
 - Compare quantity with product threshold
4. If stock is low → add alert
5. Enrich alert with supplier information
6. Return structured response

Data Model Assumed:

- Company(id)
- Warehouse(id, company_id, name)
- Product(id, name, sku, low_stock_threshold)
- Inventory(id, product_id, warehouse_id, quantity)
- InventoryHistory(id, product_id, warehouse_id, change_qty, timestamp)
- Supplier(id, name, contact_email)
- ProductSupplier(product_id, supplier_id)